# Run Grafana locally on Windows: create, preserve, and restore your first dashboard

Tim Suleyman | Technical writing portfolio | Tested October 27, 2026

Create a local Grafana dashboard with generated sample data, verify that it survives a container restart, and export and import its configuration. Along the way, diagnose two problems encountered during testing: a Docker command that Windows cannot find and a browser connection refused error.

## Audience and outcome

This tutorial is for Windows users who can run PowerShell commands but are new to Grafana. By the end, you will have a saved time-series panel and a JSON export that you have successfully imported as a separate dashboard.

The graph uses Random Walk sample data. It does not measure your computer, application performance, or a production system. Its values can change when refreshed or imported.

## Before you begin

You need:

• A Windows PC that supports Docker Desktop with the WSL 2 backend.

• WSL 2 installed and Docker Desktop running with Linux containers.

• PowerShell, a browser, and an internet connection to download images.

• Port 3000 available on your PC.

If necessary, follow the official WSL installation instructions and Docker Desktop installation instructions. Restart Windows when the installer requests it. In PowerShell, `wsl --list --verbose` should show your Linux distribution using version 2.

All commands below run in PowerShell. The tested environment used Ubuntu on WSL 2, Docker Desktop, Docker CLI 29.9.2, and Grafana 13.3.0. Grafana interface labels can differ between releases.

## 1. Verify Docker

Start Docker Desktop and wait for its engine to run. Open PowerShell and enter:

```
docker --version
docker run --rm hello-world
```

The first command should print a Docker version. The second should print `Hello from Docker!` after downloading a small test image. The `--rm` option removes the test container when it finishes.

### If PowerShell cannot find Docker

During testing, PowerShell returned `The term 'docker' is not recognized`. Docker Desktop was running, but its command directory was missing from that PowerShell session's PATH, the list of directories Windows searches for commands.

The tested per-user installation placed Docker here:

```
& "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin\docker.exe" run --rm hello-world
```

If that path exists on your PC, add its directory to the current session:

```
$env:Path = "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin;$env:Path"
docker --version
```

This change lasts only for the current PowerShell session. Installation paths vary. If the file is elsewhere, use your actual installation path instead. Closing and reopening PowerShell after installation may also resolve a stale PATH.

## 2. Start Grafana with persistent storage

For the tested setup, run:

```
docker run -d --name grafana-portfolio -p 127.0.0.1:3000:3000 -v grafana-portfolio-data:/var/lib/grafana grafana/grafana
```

Docker downloads the image if necessary and prints a container ID. Allow Grafana time to start before opening the browser.

The options have these purposes:

| Option | Purpose |
|---|---|
| `-d` | Run the container in the background. |
| `--name grafana-portfolio` | Give the container a name for later commands. |
| `-p 127.0.0.1:3000:3000` | Make Grafana available through port 3000 on this PC's IPv4 loopback address. |
| `-v grafana-portfolio-data:/var/lib/grafana` | Store Grafana's application data in a named Docker volume. Docker creates the volume if needed. |
| `grafana/grafana` | Use the Grafana Open Source image. Without a tag, Docker selects `latest`. |

Grafana documents `/var/lib/grafana` as the data location to mount for persistent storage. A volume is separate from a dashboard JSON export: the volume stores application data; the export describes an individual dashboard.

The test used `latest`, which can change. The exact repository reference recorded from the installed image was:

```
grafana/grafana@sha256:488984b716c0862639f91a698adbe2862677a5a07bad42da5f03f3030883b205
```

For a repeatable installation, substitute that complete reference for `grafana/grafana` in the command. This digest was inspected on the running installation; a separate fresh installation using the digest was not performed for this sample.

Open http://localhost:3000. On a fresh installation, sign in with username `admin` and password `admin`, then set a new password when prompted.

Verify the application version:

```
(Invoke-RestMethod -Uri "http://localhost:3000/api/health").version
```

The tested installation returned `13.3.0`.

## 3. Create a dashboard with generated data

1. Select Dashboards from the left menu.
2. Select New, then New dashboard.
3. Choose Auto grid if prompted for a layout.
4. Add a panel and select Configure visualization.
5. Select the built-in `-- Grafana --` data source.

6. In the query editor, confirm that Query type is Random Walk.

7. Choose the Time series visualization if it is not already selected.

Expected result: a line graph appears, with a series such as `A-series`. You do not need an external database for this sample.

In the panel options, enter:

• Title: `Sample data - Random Walk`

• Description: `Generated sample data for this tutorial. These values do not represent real system measurements.`

Save the panel changes and return to the dashboard. Save the dashboard as `Grafana Portfolio Lab` and exit edit mode. Depending on the editor state, use Save, Back, and Exit edit as applicable.

Refresh the browser. Confirm that the panel title remains. Hover over the information icon beside the title and confirm that the description appears.

The tested dashboard also used colored fill. Styling is optional and does not change the outcome of this exercise. Colors in this generated graph do not indicate actual system health.

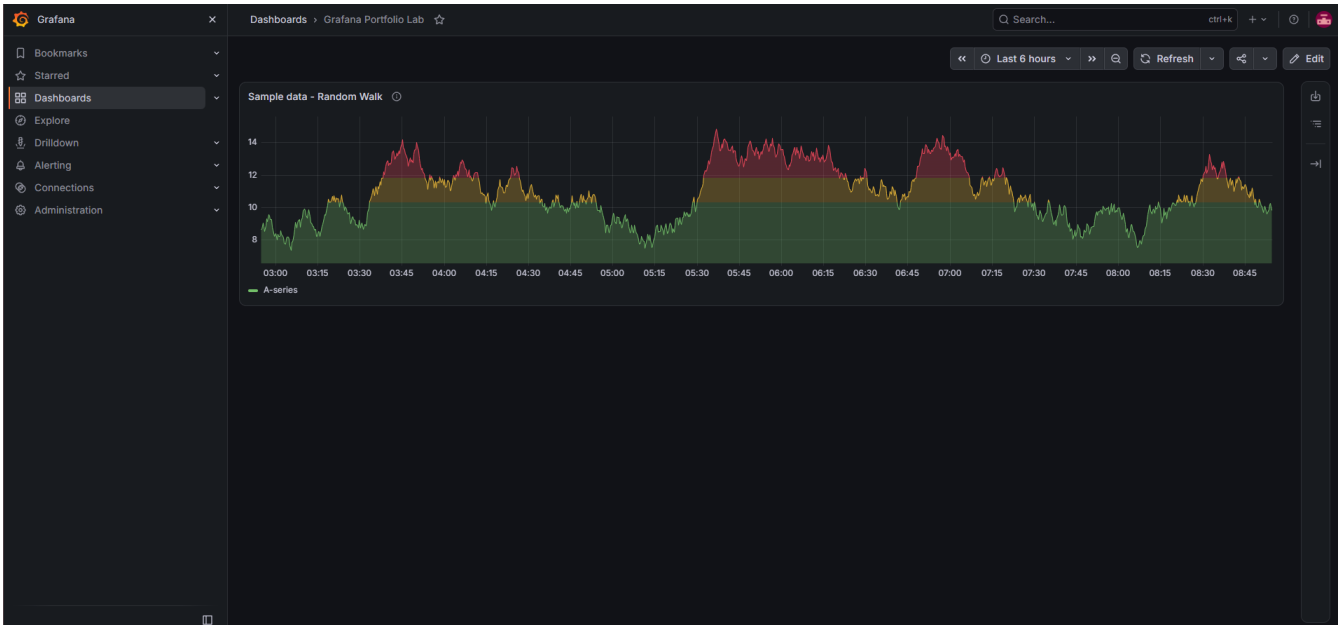



*Figure 1. Saved dashboard with generated Random Walk data. The colored fill is optional.*

## 4. Verify restart and recovery

Restart the container:

```
docker restart grafana-portfolio
```

After Grafana starts, refresh the browser. Reopen `Grafana Portfolio Lab` if necessary. Confirm that the saved panel remains.

Now reproduce a connection failure:

```
docker stop grafana-portfolio
```

Refresh the browser. During testing, it displayed `This site can't be reached` and reported that localhost refused to connect. The application is stopped, so nothing is serving Grafana on that port.

Recover it:

```
docker start grafana-portfolio
```

Wait for startup and refresh again. Expected result: Grafana is reachable and the saved dashboard remains available.

These checks verify recovery and saved configuration across a restart of the same container. They do not prove recovery after deleting a container, deleting a volume, or losing a disk.

## 5. Export the dashboard as JSON

1. Open `Grafana Portfolio Lab` in dashboard view.

2. Open the export control on the far right, shown as a downward arrow entering a tray in the tested interface.

3. Select Export as code. The Share dropdown is a different menu.

4. Copy the complete JSON representation.

5. Paste it into Notepad. Select File > Save As, set Save as type to All files, choose UTF-8 encoding, and save as `grafana-portfolio-lab.json`.

Confirm that Windows has not appended `.txt` to the filename.

The tested export began with `apiVersion: dashboard.grafana.app/v2` and `kind: Dashboard`. Its `spec` contained the dashboard title, panel title, description, layout, queries, and visualization settings.

A JSON export preserves dashboard configuration. It is not a backup of all Grafana users, passwords, data sources, or historical monitoring data. Review exports before publishing: they can include instance identifiers and query details.

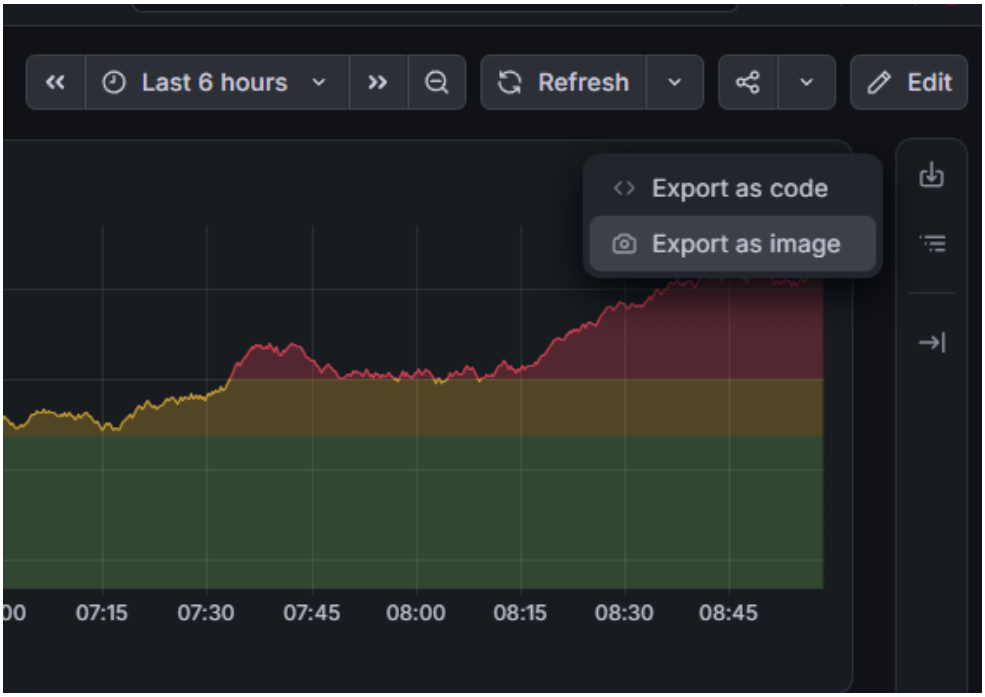



*Figure 2. Export as code is in the export menu, separate from sharing options.*

## 6. Import a separate copy

1. Select Dashboards > New > Import.

2. Load `grafana-portfolio-lab.json` through the file option, or paste its complete contents into the JSON input and load it.

3. On the import configuration screen, change Name to `Grafana Portfolio Lab - Import Test`.

4. Select Change uid and set a different identifier, such as `portfolio-import-test`, if prompted. A UID identifies a dashboard independently of its displayed name.

5. Confirm that the duplicate-name and duplicate-UID warnings disappear. The action should no longer say Import (Overwrite).

6. Select Import.

Expected result: the imported dashboard opens as a separate copy. Check all four items:

• Dashboard title: `Grafana Portfolio Lab - Import Test`.

• Panel title: `Sample data - Random Walk`.

• The information icon displays the original description.

• A time-series graph displays sample data.

The generated values do not need to match the original graph. In this test, the graph, panel title, and description all appeared after import.

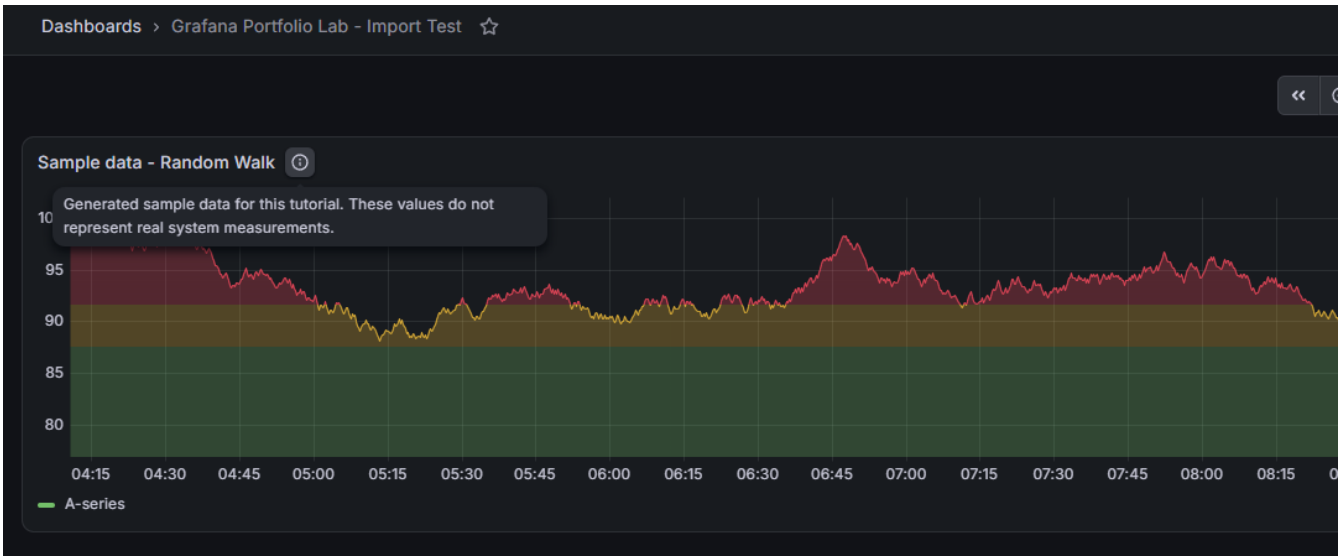



*Figure 3. Imported dashboard with the restored panel title, description, and graph.*

## Troubleshooting

| Symptom | Check and next action |
|---|---|
| `docker` is not recognized | Follow the PATH workaround in step 1 using the actual Docker installation directory. |
| Browser reports connection refused | Confirm Docker Desktop is running. Run `docker start grafana-portfolio`, allow startup time, and refresh. This resolved the deliberately stopped-container test. |
| Import reports an existing dashboard name or UID | Change both the imported name and UID to create a separate copy. Check that the button no longer offers overwrite. |
| Imported graph differs from the original | Random Walk generates sample values. Verify the title, description, and working visualization instead of identical numbers. |

## Test record and limits

| Check | Observed result |
| --- | --- |
| Docker verification | `hello-world` completed successfully. |
| Application version | Health endpoint returned Grafana 13.3.0. |
| Save and browser refresh | Panel title and description remained. |
| Container restart | Saved dashboard remained available. |
| Stopped container | Browser reported connection refused. |
| Container started again | Grafana and the saved dashboard returned. |
| JSON export | File parsed successfully as JSON. |
| Import with a new name and UID | Separate dashboard displayed the graph, title, and description. |

Tested on one personal Windows environment on October 27, 2026. The checks above were performed by the author during a guided session. Fresh-machine reproduction, volume recovery after container removal, and compatibility with other Grafana versions were not tested. This exercise demonstrates local setup and dashboard configuration, not production observability or disaster recovery.

## References


- Grafana: Run Grafana Docker image
- Grafana: Build your first dashboards
- Grafana: Share dashboards and panels
- Microsoft: Install WSL
- Docker: Install Docker Desktop on Windows


Interface instructions reflect the observed Grafana 13.3.0 session. Linked documentation may describe a different release.